

Universidade Federal de Santa Catarina

21/06/2024, Araranguá - SC

Trabalho de Banco de Dados:

LUNI

**Plataforma de gestão de
ingressos**

Lucas Porto (22103587)

Nikolas Lopes (19104066)

Disciplina de Banco de Dados I
(DEC7129)

Prof. Alexandre Leopoldo Gonçalves

Sumário

Sumário.....	2
1. Objetivo geral do sistema.....	3
2. Descrição Detalhada.....	3
3. Modelagem conceitual.....	5
4. Modelagem lógica.....	6
5. Script DDL.....	6
6. Consultas.....	10
6.1. Consulta 1.....	10
6.2. Consulta 2.....	11
6.3. Consulta 3.....	13

1. Objetivo geral do sistema

O objetivo geral deste sistema de venda de ingressos é oferecer uma plataforma onde os usuários possam facilmente explorar e adquirir ingressos para uma ampla variedade de eventos, dependendo da sua preferência. Além disso, os usuários poderão visualizar informações detalhadas sobre os eventos. O sistema visa fornecer uma plataforma confiável que permita aos usuários descobrir e acessar eventos de maneira personalizada, enquanto garante a eficiência na gestão e processamento de pedidos por meio de uma infraestrutura de banco de dados.

2. Descrição Detalhada

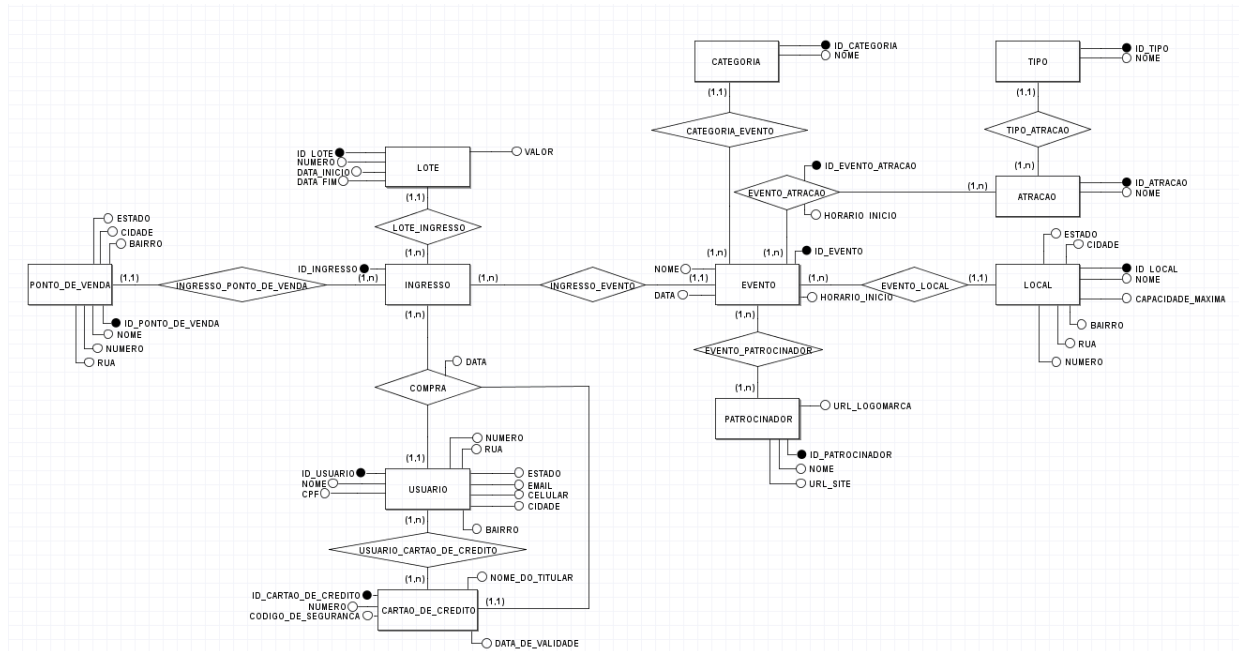
Dentre os requisitos do sistema, estão:

- A identificação dos usuários: cada usuário irá possuir um e-mail, um nome e um CPF que estarão salvos no banco de dados. Essas informações serão armazenadas na tabela USUARIO, que também inclui campos para celular, endereço (estado, bairro, cidade, rua, número) e uma chave primária única para cada usuário.
- Compra De Ingressos : os usuários do serviço podem comprar ingressos pelo site, ou podem se deslocar-se a uma unidade de venda física, na qual estará especificado no site, o endereço de uma loja parceira que possa vender os ingressos, na tabela consta (nome, bairro, cidade, estado, rua, número).
- Categorização dos eventos e atrações: os eventos devem possuir um nome, data e horário de início e devem estar relacionados a uma categoria específica. Ademais, cada evento pode ter diversas atrações (cantor, banda, músico, palestrante) subcategorizadas por um tipo que podem participar de diversos eventos.

- Os ingressos possuem lote: O lote depende da data de início ou fim, de acordo com isso os valores sofrem alterações e o número do lote sobe, quanto maior o número do lote, mais caro o ingresso fica.
- Local do evento: Cada evento possui um local, e um mesmo local pode pertencer a vários eventos, como atributos do local temos (nome, capacidade máxima, bairro, rua, cidade, estado, número).
- Patrocinadores: Cada evento pode ser patrocinado por diversos patrocinadores, e estes podem patrocinar vários eventos. Os patrocinadores devem possuir uma logomarca e seu site para ser exibido no site de venda de ingressos.

3. Modelagem conceitual

Figura 1 - Modelagem Conceitual



4. Modelagem lógica

Figura 2 - Modelagem Lógica



5. Script DDL

```
CREATE TABLE PONTO_DE_VENDA (  
  id_ponto_de_venda integer PRIMARY KEY NOT NULL,  
  nome varchar(50) NOT NULL,
```

```
    bairro varchar(50) NOT NULL,
    cidade varchar(50) NOT NULL,
    estado varchar(50) NOT NULL,
    rua varchar(50) NOT NULL,
    numero integer NOT NULL
);

CREATE TABLE LOTE (
    id_lote integer PRIMARY KEY NOT NULL,
    numero integer NOT NULL,
    data_fim timestamp NOT NULL,
    valor float NOT NULL,
    data_inicio timestamp NOT NULL
);

CREATE TABLE CATEGORIA (
    id_categoria integer PRIMARY KEY NOT NULL,
    nome varchar(50) NOT NULL
);

CREATE TABLE LOCAL (
    id_local integer PRIMARY KEY NOT NULL,
    nome varchar(50) NOT NULL,
    capacidade_maxima integer NOT NULL,
    bairro varchar(50) NOT NULL,
    rua varchar(50) NOT NULL,
    cidade varchar(50) NOT NULL,
    estado varchar(50) NOT NULL,
    numero integer NOT NULL
);

CREATE TABLE TIPO (
    id_tipo integer PRIMARY KEY NOT NULL,
    nome varchar(50) NOT NULL
);

CREATE TABLE CARTAO_DE_CREDITO (
    id_cartao_de_credito integer PRIMARY KEY NOT NULL,
    nome_do_titular varchar(50) NOT NULL,
    codigo_de_seguranca integer NOT NULL,
    numero varchar(50) NOT NULL,
    data_de_validade timestamp NOT NULL,
    UNIQUE(numero)
);

CREATE TABLE USUARIO (
    id_usuario integer PRIMARY KEY NOT NULL,
    email varchar(50) NOT NULL,
```

```

        celular varchar(50) NOT NULL,
        cpf varchar(50) NOT NULL,
        nome varchar(50) NOT NULL,
        estado varchar(50) NOT NULL,
        bairro varchar(50) NOT NULL,
        cidade varchar(50) NOT NULL,
        rua varchar(50) NOT NULL,
        numero integer NOT NULL,
        UNIQUE(cpf)
    );

CREATE TABLE PATROCINADOR (
    id_patrocinador integer PRIMARY KEY NOT NULL,
    url_logomarca varchar(100) NOT NULL,
    url_site varchar(100) NOT NULL,
    nome varchar(50) NOT NULL
);

CREATE TABLE USUARIO_CARTAO_DE_CREDITO (
    id_cartao_de_credito integer NOT NULL,
    id_usuario integer NOT NULL,
    PRIMARY KEY(id_cartao_de_credito, id_usuario),
    FOREIGN KEY(id_cartao_de_credito) REFERENCES CARTAO_DE_CREDITO
(id_cartao_de_credito),
    FOREIGN KEY(id_usuario) REFERENCES USUARIO (id_usuario)
);

CREATE TABLE ATRACAO (
    id_atracao integer PRIMARY KEY NOT NULL,
    nome varchar(50) NOT NULL,
    id_tipo integer NOT NULL,
    FOREIGN KEY(id_tipo) REFERENCES TIPO (id_tipo)
);

CREATE TABLE EVENTO (
    id_evento integer PRIMARY KEY NOT NULL,
    data timestamp NOT NULL,
    nome varchar(50) NOT NULL,
    horario_inicio time NOT NULL,
    id_local integer NOT NULL,
    id_categoria integer NOT NULL,
    FOREIGN KEY(id_local) REFERENCES LOCAL (id_local),
    FOREIGN KEY(id_categoria) REFERENCES CATEGORIA (id_categoria)
);

CREATE TABLE EVENTO_ATRACAO (
    id_evento_atracao integer PRIMARY KEY NOT NULL,
    horario_inicio time NOT NULL,
    id_evento integer NOT NULL,

```



```

        id_atracao integer NOT NULL,
        FOREIGN KEY(id_evento) REFERENCES EVENTO (id_evento),
        FOREIGN KEY(id_atracao) REFERENCES ATRACAO (id_atracao)
    );

CREATE TABLE EVENTO_PATROCINADOR (
    id_patrocinador integer NOT NULL,
    id_evento integer NOT NULL,
    PRIMARY KEY(id_patrocinador, id_evento),
    FOREIGN KEY(id_patrocinador) REFERENCES PATROCINADOR (id_patrocinador),
    FOREIGN KEY(id_evento) REFERENCES EVENTO (id_evento)
);

CREATE TABLE INGRESSO (
    id_ingresso integer PRIMARY KEY NOT NULL,
    id_ponto_de_venda integer NOT NULL,
    id_lote integer NOT NULL,
    id_evento integer NOT NULL,
    FOREIGN KEY(id_ponto_de_venda) REFERENCES PONTO_DE_VENDA
(id_ponto_de_venda),
    FOREIGN KEY(id_lote) REFERENCES LOTE (id_lote),
    FOREIGN KEY(id_evento) REFERENCES EVENTO (id_evento)
);

CREATE TABLE COMPRA (
    id_ingresso integer NOT NULL,
    id_cartao_de_credito integer NOT NULL,
    id_usuario integer NOT NULL,
    data timestamp NOT NULL,
    PRIMARY KEY(id_ingresso, id_cartao_de_credito, id_usuario),
    FOREIGN KEY(id_ingresso) REFERENCES INGRESSO (id_ingresso),
    FOREIGN KEY(id_cartao_de_credito) REFERENCES CARTAO_DE_CREDITO
(id_cartao_de_credito),
    FOREIGN KEY(id_usuario) REFERENCES USUARIO (id_usuario)
)

```

6. Consultas

6.1. Consulta 1

Exibir o valor total arrecadado com a venda de ingressos para cada evento.

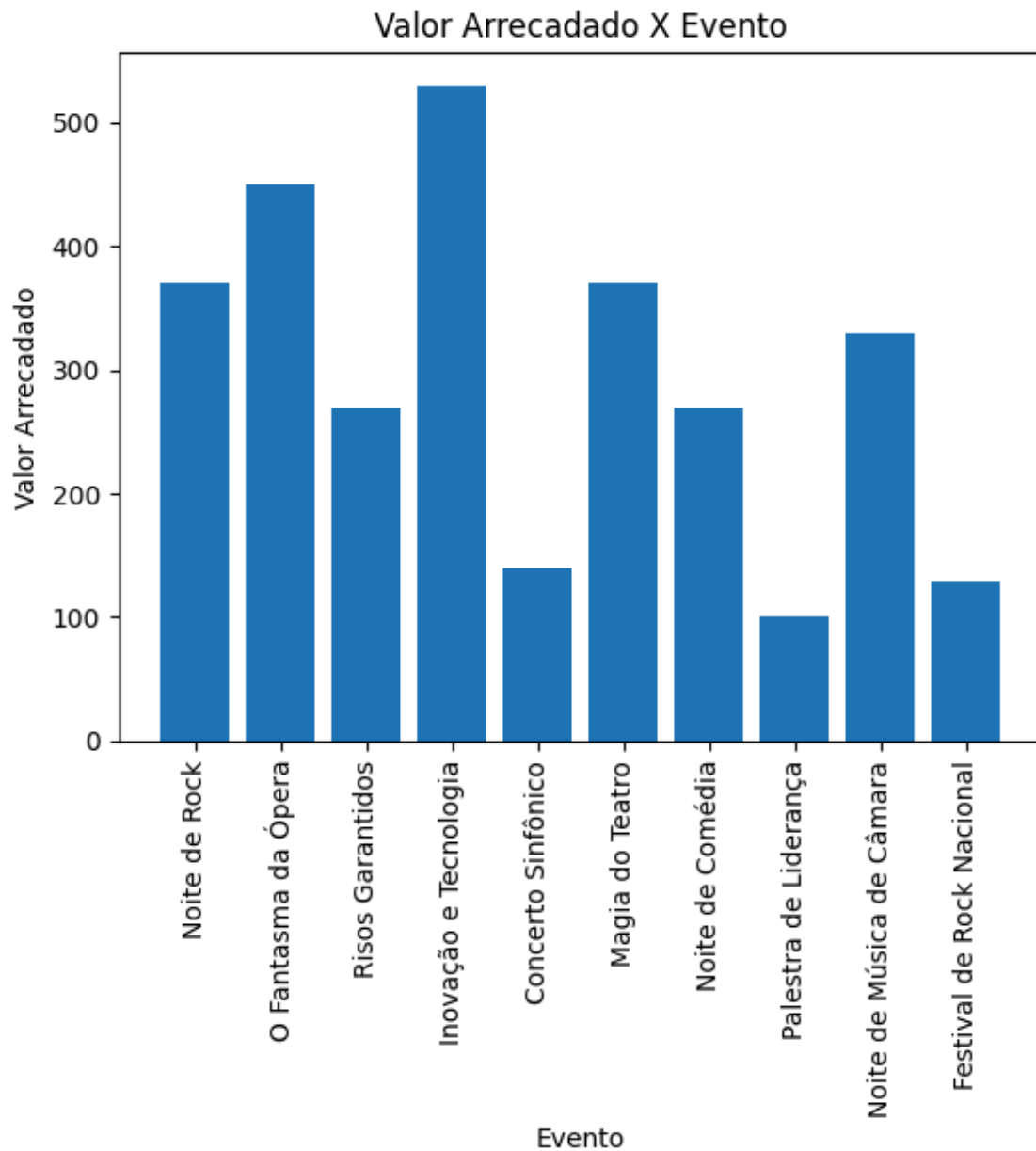
SQL:

```
SELECT
    EVENTO.id_evento,
    EVENTO.nome,
    sum(valor)
FROM
    EVENTO,
    INGRESSO,
    LOTE,
    COMPRA
WHERE
    EVENTO.id_evento = INGRESSO.id_evento AND
    INGRESSO.id_ingresso = COMPRA.id_ingresso AND
    INGRESSO.id_lote = LOTE.id_lote
GROUP BY
    EVENTO.id_evento
ORDER BY
    EVENTO.id_evento;
```

Resultado:

```
(1, 'Noite de Rock', 370.0)
(2, 'O Fantasma da Ópera', 450.0)
(3, 'Risos Garantidos', 270.0)
(4, 'Inovação e Tecnologia', 530.0)
(5, 'Concerto Sinfônico', 140.0)
(6, 'Magia do Teatro', 370.0)
(7, 'Noite de Comédia', 270.0)
(8, 'Palestra de Liderança', 100.0)
(9, 'Noite de Música de Câmara', 330.0)
(10, 'Festival de Rock Nacional', 130.0)
```

Figura 3 - Consulta 1



6.2. Consulta 2

Mostrar o número de atrações em cada evento.

SQL:

```
SELECT
    EVENTO.id_evento,
    EVENTO.nome,
    count(*)
FROM
    EVENTO,
    EVENTO_ATRACAO,
    ATRACAO
```

```
WHERE
    EVENTO.id_evento = EVENTO_ATRACAO.id_evento AND
    ATRACAO.id_atracao = EVENTO_ATRACAO.id_atracao
GROUP BY
    EVENTO.id_evento
ORDER BY
    count(*) DESC;
```

Resultado:

```
(1, 'Noite de Rock', 2)
(2, 'O Fantasma da Ópera', 3)
(3, 'Risos Garantidos', 3)
(4, 'Inovação e Tecnologia', 3)
(5, 'Concerto Sinfônico', 3)
(6, 'Magia do Teatro', 3)
(7, 'Noite de Comédia', 3)
(8, 'Palestra de Liderança', 3)
(9, 'Noite de Música de Câmara', 3)
(10, 'Festival de Rock Nacional', 2)
```

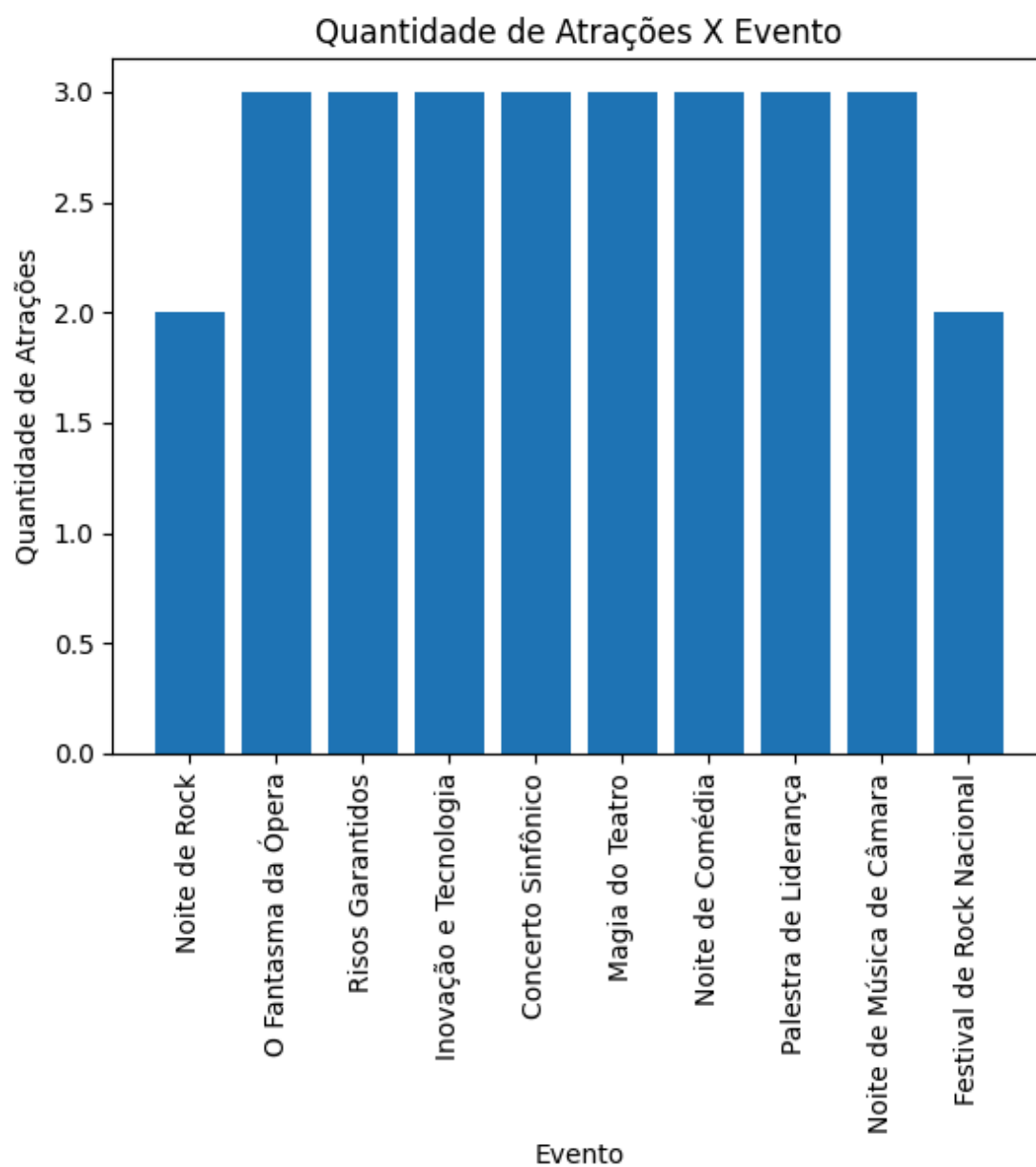


Figura 4 - Consulta 3

6.3. Consulta 3

Exibir a quantidade de ingressos que restam ser vendidos para atingir a capacidade máxima de lotação em cada evento.

SQL:

```
SELECT
    EVENTO.id_evento,
    EVENTO.nome,
    LOCAL.capacidade_maxima - count(*)
FROM
    EVENTO,
    INGRESSO,
    COMPRA,
    LOCAL
WHERE
    EVENTO.id_evento = INGRESSO.id_evento AND
    INGRESSO.id_ingresso = COMPRA.id_ingresso AND
    EVENTO.id_local = LOCAL.id_local
GROUP BY
    EVENTO.id_evento, EVENTO.nome
ORDER BY
    EVENTO.id_evento;
```

Resultado:

```
(1, 'Noite de Rock', 14997)
(2, 'O Fantasma da Ópera', 1198)
(3, 'Risos Garantidos', 297)
(4, 'Inovação e Tecnologia', 4997)
```

- (5, 'Concerto Sinfônico', 14999)
- (6, 'Magia do Teatro', 1197)
- (7, 'Noite de Comédia', 297)
- (8, 'Palestra de Liderança', 4999)
- (9, 'Noite de Música de Câmara', 798)
- (10, 'Festival de Rock Nacional', 14999)

Figura 4 - Consulta 3

